User Manual

Running the platform, and what each screen does

---

1. Starting up

On Windows, `run_application.bat` creates the virtual environment, installs dependencies, and starts both processes. Manually:

```
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # macOS / Linux
pip install -r requirements.txt
cp .env.example .env

uvicorn app.main:app --port 8000 --reload # terminal 1
streamlit run frontend/app.py # terminal 2
```

- Web portal: `http://localhost:8501`
- API and interactive docs: `http://127.0.0.1:8000/docs`
- Health probe: `http://127.0.0.1:8000/health`

On first start the database is created, the demo accounts are seeded, and the catalog is populated with the PayPal collection, the general-purpose tools, the two pinned meta-tools, and the built-in MCP tools.

2. Accounts

Role	Email	Password	Access
Administrator	admin@examplemail.com	admin@123	Everything: chat, tool creation, document ingestion, and every user's activity
Standard user	user@examplemail.com	user@123	Chat, browse active tools and documents, and only their own activity

LIMITATION
These are seeded development credentials. Change them, and JWT_SECRET_KEY, before exposing the platform to anyone else.

3. Screen 1 - Chat

Ask in plain language. The pipeline panel lists the stages that actually ran for your turn; expanding a reply shows the tools executed, the tokens consumed, and the full execution trace with arguments and responses.

Try	What happens
What tools are available for managing invoices?	System Search returns matching catalog entries
What are the valid refund reason codes?	RAG retrieves from the knowledge base and cites the document
What was my total sales volume last month?	Retrieves the transaction report and reports the returned figures
Is there a dispute open from user_123?	Lists disputes, since no specific dispute identifier was supplied
Send an invoice for $50 to client@example.com	Pauses for confirmation, because sending mutates external state

Try	What happens
What is the status of my last request?	System Search returns your own recent executions

Confirming an action

When an action changes external state the run pauses and shows the tool, the reason, and the exact arguments that would be sent. **Approve** resumes the run and continues through any remaining steps; **Reject** cancels it and nothing is dispatched. A decision can be made once.

4. Screen 2 - Tools and Sources

Registered tools

Browse and filter the catalog by category, method and status. Administrators can register a tool through a form covering basic information, API configuration, path, query and header parameters, request and response schemas, execution behaviour (mutating, approval required, permitted roles, retries), prerequisite dependencies, and schema validation with a mock test run. A registered tool is written to both the relational store and the vector index, so it becomes discoverable immediately.

Documents and knowledge base

Administrators can ingest a local file (PDF, TXT, MD, CSV, JSON), a documentation URL, or pasted text. Content is extracted, chunked with overlap, embedded and indexed with metadata, after which the RAG tool can answer from it. The listing shows each source's domain, type and chunk count.

MCP servers

Discover the tools an MCP server exposes and index them into the catalog, after which they are retrieved and executed like any other tool.

5. Screen 3 - Usage

A standard user sees their own totals and a table of their tasks: what was asked, whether it succeeded, how many tools ran, the prompt, output and total tokens for that task, and its latency. An administrator sees platform totals and a user directory; expanding a user shows the same per-task table for that account, plus the tools they have dispatched.

ON TOKEN COUNTS
Token counts are the figures the model SDK reported. With no API key configured no model runs, so the totals are legitimately zero rather than estimated.

6. Screen 4 - Logs

A standard user sees only their own tasks. An administrator picks a user and expands them to see that user's tasks, each with its tool calls in a table: tool, description or error, status, latency and timestamp. Selecting a call shows the input arguments and the response payload. Failed calls are included, which is the point - the log is an audit trail, not a success feed.

7. Troubleshooting

Symptom	Cause and fix
ImportError about greenlet on startup	SQLAlchemy was installed without the asyncio extra. Reinstall from requirements.txt.
Token totals are zero	No GEMINI_API_KEY is configured, so no model ran. Expected; set the key to use the model.
Replies look templated and terse	The deterministic fallback planner is active, for the same reason.
Local PDF upload fails	pypdf is missing. Reinstall from requirements.txt.
Frontend shows a connection error	The backend is not running, or is on a different port than the client expects.
A run cannot be approved and returns 409	It was already decided, or its checkpoint is gone after a restart in in-memory mode. Re-send the request.
The catalog looks empty	Seeding runs on first start; check the backend log for the auto-seed line.